Karadeniz Teknik Üniversitesi

Fen Fakültesi – Bilgisayar Bilimleri Bölümü

Qt Framework'üne Giriş

Bölüm 4 – Hafta 4

Yazılım Geliştirme II (BILB2000)

Bahar Dönemi – 4. Hafta

Bu Haftanın Planı

1. Ünite 1: Qt Ekosistemi
2. Ünite 2: QObject ve Meta-Object Sistemi
3. Ünite 3: Sinyaller ve Yuvalar
4. Ünite 4: Temel Widget'lar

Ünite 1: Qt Ekosistemi

Qt Nedir?

Tanım

Qt ("cute" olarak okunur), C++ ile **çapraz platform** uygulamalar geliştirmek için kullanılan açık kaynak bir framework'tür. Windows, macOS, Linux, Android ve iOS için tek kod tabanıyla uygulama geliştirmeyi mümkün kılar.

Qt'nin Sunduğu Yetenekler:

- GUI (pencere, buton, form)
- Ağ programlama (TCP/UDP, HTTP)
- Veritabanı erişimi (SQLite, PostgreSQL)
- Çok kanallı programlama (QThread)
- Dosya sistemi, JSON, XML

OBYS Bağlamı

Konsol OBYS'mizden masaüstü OBYS'ye geçiyoruz! STL tabanlı veri katmanımızın üzerine görsel bir arayüz inşa edeceğiz.

Qt'nin Kısa Tarihçesi

Qt'nin hikayesi 1991'de Norveç'te başlar:

1. **1995 – Qt 1.0**: İlk sürüm, X11 (Linux) ve Windows desteği, Trolltech şirketi kuruldu
2. **1996 – KDE**: Linux masaüstü ortamı KDE, Qt üzerine kuruldu
3. **2005 – Qt 4.0**: Tamamen yeniden yazıldı; Arthur (2D grafik), Interview (model/view)
4. **2008 – Nokia**: Nokia, Trolltech'i satın aldı
5. **2012 – Qt Company**: Qt, bağımsız Qt Company'ye devredildi
6. **2012 – Qt 5.0**: QML ve Qt Quick desteği, modüler mimari
7. **2020 – Qt 6.0**: C++17 zorunlu, CMake birincil derleme sistemi, yeni grafik mimarisi (RHI)

Bu Kitapta

Qt 6 + C++20 + CMake kullanacağız. Qt 5'ten Qt 6'ya geçişte API'lerin büyük çoğunluğu korunmuştur.

Qt Lisans Modelleri

Açık Kaynak (LGPL v3 / GPL v3)

- Ücretsiz kullanım
- Kaynak kodu paylaşma yükümlülüğü
- Akademik ve kişisel projeler için ideal
- Tüm modüllere erişim
- **Biz bunu kullanacağız**

Ticari Lisans

- Ücretli (yıllık abonelik)
- Kaynak kodu paylaşma zorunluluğu yok
- Kapalı kaynak ürünlerde tercih edilir
- Özel destek ve eğitim

Üniversite Projelerinde

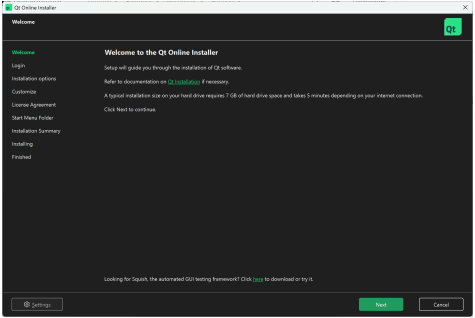
LGPL lisansı yeterli. Qt'nin tüm modüllerine ücretsiz erişebilirsin.

Qt Kurulumu – Platform Karşılaştırması

	Windows	Linux
Derleyici	MinGW / MSVC	GCC
Kurulum	Qt Installer	`apt install`
IDE	Qt Creator	Qt Creator
Disk	4–15 GB	1–3 GB

Qt Online Installer

`qt.io` → hesap oluştur → "Open Source" seç → Qt 6.x + Kit (MinGW/MSVC) → Qt Creator + CMake + Ninja



Qt Online Installer – Hoş Geldiniz ekranı

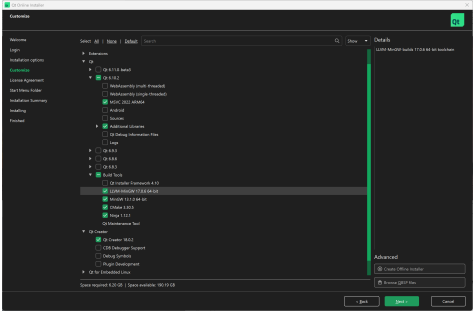
Qt Kurulumu – Bileşen Seçimi

Kurulum sırasında seçilecek bileşenler:

1. **Qt 6.x** – Ana framework
2. **Kit** – MinGW (Windows) veya GCC (Linux)
3. **Qt Creator** – IDE
4. **CMake + Ninja** – Derleme araçları

Disk Alanı

Tam kurulum 4–15 GB yer kaplar. Sadece ihtiyacın olan modülleri seç!



Bileşen seçim ekranı

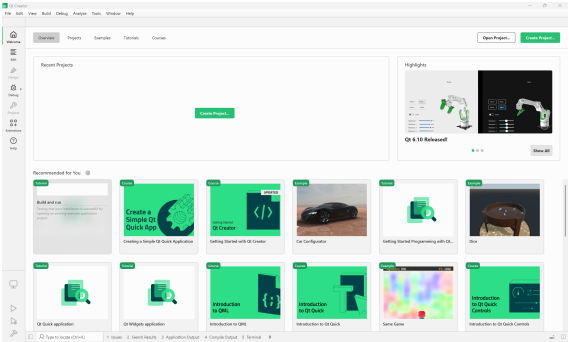
Qt Creator IDE

Qt Creator özellikleri:

- **Kod editörü**: Sözdizimi renklendirme, otomatik tamamlama
- **Görsel tasarımcı**: Qt Designer – sürükle-bırak
- **Proje yönetimi**: CMake desteği
- **Hata ayıklama**: GDB/LLDB/CDB
- **Profilleme**: Valgrind

Alternatif IDE'ler

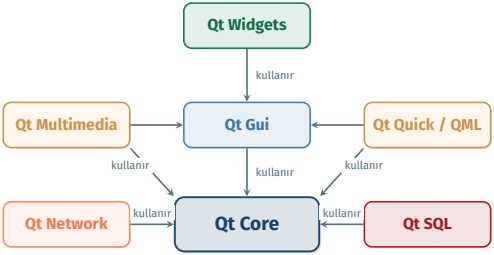
Visual Studio, CLion, VS Code



Qt Creator karşılama ekranı

Qt Modülleri

Qt 6 Modül Ekosistemi



Oklar bağımlılık yönünü gösterir: üst modüller alt modüllere bağımlıdır

Temel Modüller:

- `Qt Core` – sinyal/slot, QObject
- `Qt Gui` – 2D grafik, pencere
- `Qt Widgets` – buton, etiket, form
- `Qt Network` – TCP/UDP, HTTP

Ek Modüller:

- `Qt SQL` – veritabanı erişimi
- `Qt Multimedia` – ses/video
- `Qt Quick` – QML arayüz
- `Qt Charts` – grafik/çizelge

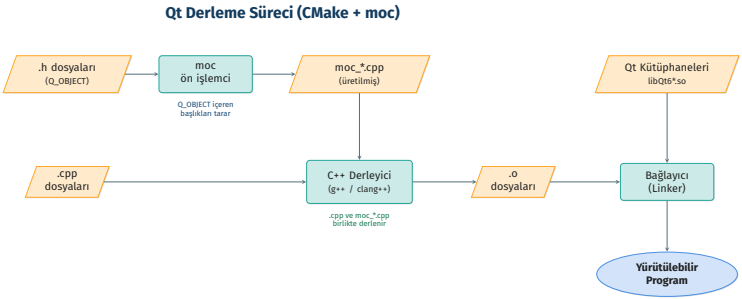
CMake ile Qt Projesi

```
cmake_minimum_required(VERSION 3.16)
project(MerhabaQt VERSION 1.0 LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

find_package(Qt6 REQUIRED COMPONENTS Widgets)
qt_standard_project_setup()        # moc, uic, rcc otomatik

add_executable(merhaba_qt main.cpp)
target_link_libraries(merhaba_qt PRIVATE Qt6::Widgets)
```

Anahtar Fonksiyonlar

`find_package` → Qt'yi bul `qt_standard_project_setup` → moc/uic/rcc otomatik
`target_link_libraries` → kütüphane bağla

CMake Derleme Süreci



Komut Satırından Derleme

`mkdir build && cd build` → `cmake ..` → `cmake -build .` → `./merhaba_qt`

find_package Hatası Alırsan

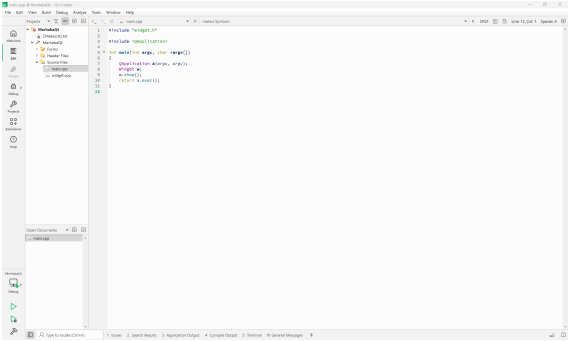
```
cmake -DCMAKE_PREFIX_PATH=/path/to/Qt/6.x ..
```

Qt Projesi Dosya Yapısı

```
proje_dizini/
+-- CMakeLists.txt
+-- main.cpp
+-- mainwindow.h
+-- mainwindow.cpp
+-- mainwindow.ui
+-- resources.qrc
+-- build/
```

Şimdilik

Yalnızca `CMakeLists.txt` ve `main.cpp` ile çalışacağız.



Qt Creator'da CMake projesi

İlk Qt Uygulaması: Merhaba Qt!

```
#include <QApplication>
#include <QLabel>

int main(int argc, char *argv[]) {
    QApplication uygulama(argc, argv);

    QLabel etiket("Merhaba Qt!");
    etiket.setStyleSheet(
        "font-size: 24px; padding: 20px;");
    etiket.setAlignment(Qt::AlignCenter);
    etiket.resize(300, 100);
    etiket.show();

    return uygulama.exec();
}
```

Çalışır hali

Kavramlar

`QApplication` → olay döngüsü
`show()` → görünür yap
`exec()` → döngüye gir

Qt Modülleri ve #include

Sınıf Bazlı (önerilen):

```
#include <QApplication>
#include <QWidget>
#include <QPushButton>
#include <QLabel>
#include <QLineEdit>
#include <QVBoxLayout>
```

Her sınıf kendi başlık dosyasına sahip. Derleme süresi kısa.

Modül Bazlı:

```
#include <QtWidgets>
// Tum Widgets siniflarini
// dahil eder
```

Tek satır ile tüm widget'lar. Derleme süresini artırır.

Performans Notu

`#include <QtWidgets>` tüm başlık dosyalarını dahil eder → büyük projelerde derleme süresini artırır. Yalnızca ihtiyacın olan sınıfları tek tek dahil et.

Ünite 2: QObject ve Meta-Object Sistemi

QObject: Temel Sınıf

Tanım

QObject, Qt nesne modelinin temel sınıfıdır. Sinyal/slot mekanizması, parent-child bellek yönetimi, olay işleme ve çalışma zamanı tür bilgisi bu sınıf üzerine kurulmuştur.

QObject'in Yetenekleri:

- **Sinyal/Slot**: Nesneler arası iletişim (Ünite 3)
- **Parent-Child**: Otomatik bellek yönetimi
- **Olay İşleme**: Fare, klavye, zamanlama olayları
- **Meta-Object**: Çalışma zamanı tür sorgulama (`className()`, `inherits()`)
- **Property Sistemi**: `Q_PROPERTY` ile dinamik özellikler

OBYS'de

Arayüz ve yönetici sınıfları (`AnaForm`, `OgrenciYonetici`) QObject'ten türer. Veri sınıfları (`Ogrenci`) düz C++ struct kalır.

Q_OBJECT Makrosu

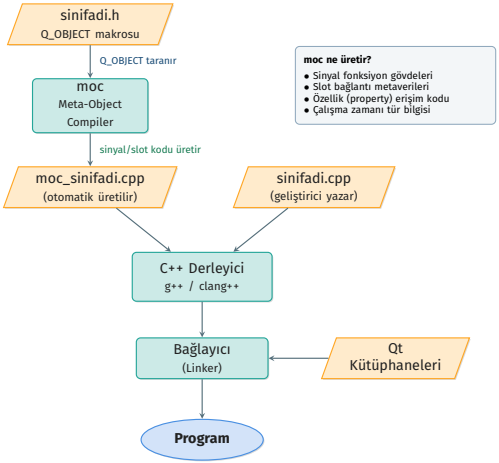
```
#include <QObject>

class OgrenciYonetici : public QObject {
    Q_OBJECT  // Bu makro ZORUNLU!

public:
    explicit OgrenciYonetici(QObject *parent = nullptr);

signals:
    void ogrenciEklendi(const QString &isim);

public slots:
    void ogrenciEkle(const QString &isim);
};
```

Dikkat

Q_OBJECT makrosunu unutursan sinyal/slot çalışmaz! Derleme başarılı olsa bile çalışma zamanında hata alırsın. qobject_cast da başarısız olur.

Meta-Object Compiler (moc)

Meta-Object Compiler (moc) Süreci



moc ne üretir?

1. `Q_OBJECT` başlıkları tarar
2. `moc_*.cpp` dosyaları üretir
3. Sinyal/slot tablosu oluşturur
4. Normal C++ ile derlenir

CMake ile Otomatik

`qt_standard_project_setup()` moc'u otomatik çalıştırır.

QObject Kısıtlamaları

QObject Kopyalanamaz ve Taşınamaz!

Copy constructor, assignment operator ve move semantics silinmiştir.

```
QObject a;
// QObject b = a;                // DERLEME HATASI!
// QObject c(std::move(a));      // DERLEME HATASI!
```

Neden?

- Her QObject benzersiz bir kimliğe sahip (sinyal bağlantıları, çocuklar)
- Kopyalama, sahiplik ağacını bozar
- Çözüm: işaretçi veya referans kullan

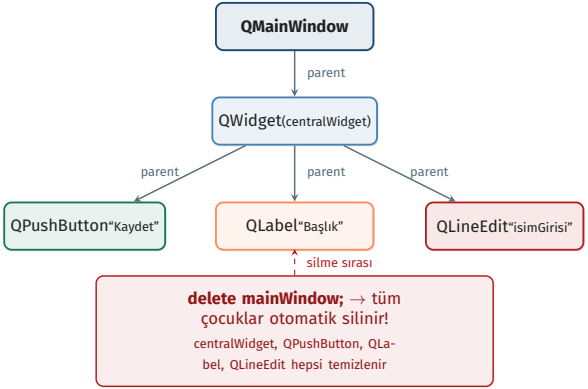
STL Kapsayıcılarda

`std::vector<QObject>` → **YAPILAMAZ!**

`std::vector<QObject*>` → Doğru yol

Parent-Child Sahiplik Modeli

QObject Ebeveyn-Çocuk Ağacı



Tanım

Her `QObject` bir ebeveyne sahip olabilir. Ebeveyn silindiğinde **tüm çocuklar** otomatik silinir.

Avantajlar:

- Bellek sızıntısı riski azalır
- Kök nesneyi silmek yeterli
- Widget'larda parent = görsel kapsayıcı

Ebeveyn Belirleme

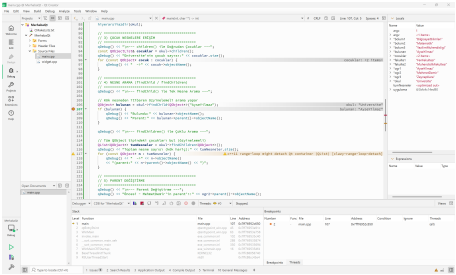
```
// Yol 1: Constructor'da ebeveyn belirt
QPushButton *buton = new QPushButton("Tikla", pencere);
// "pencere" nesnesi butonun ebeveyni olur

// Yol 2: setParent() ile sonradan belirle
QLabel *etiket = new QLabel("Merhaba");
etiket->setParent(pencere);
// Simdi "pencere" etiketin ebeveyni

// Yol 3: Layout'a ekleyince otomatik
auto *duzen = new QVBoxLayout(pencere);
auto *btn = new QPushButton("Test"); // parent yok
duzen->addWidget(btn);               // pencere parent olur
```

Ebeveyn Olmadan `new`

Ebeveyn belirtmeden `new` ile oluşturulan nesne ağaçta yalnız kalır ve otomatik silinmez. Ya kendin `delete` et ya da bir ebeveyne ata.

Nesne Ağacında Gezinme

```
QWidget *pencere = new QWidget;
pencere->setObjectName("anaPencere");

QPushButton *btn1 = new QPushButton("Kaydet", pencere);
btn1->setObjectName("kaydetButonu");

QPushButton *btn2 = new QPushButton("Iptal", pencere);
btn2->setObjectName("iptalButonu");

// Isimle cocuk bul
auto *b = pencere->findChild<QPushButton*>(
    "kaydetButonu");

// Turdeki tum cocuklari bul
auto list = pencere->findChildren<QPushButton*>();

// Cocuk listesini gez
for (auto *c : pencere->children())
    qDebug() << c->objectName();
```



Debugger'da QObject hiyerarşisi

Sahiplik Kuralları

Parent-child modelinin dört temel kuralı:

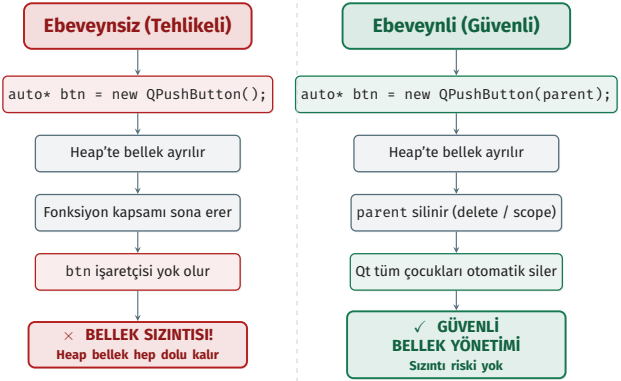
1. Bir QObject **yalnızca bir ebeveyne** sahip olabilir. Yeni ebeveyn atanırsa eski ebeveynden otomatik ayrılır.
2. Ebeveyn yok edildiğinde çocuklar **LIFO sırasıyla** (eklendikleri sıranın tersinde) yok edilir.
3. Bir çocuk ebeveyninden **önce yok edilebilir**. Bu durumda kendini ebeveynin çocuk listesinden otomatik çıkarır.
4. Widget'larda ebeveyn aynı zamanda **görsel hiyerarşiyi** belirler: çocuk widget, ebeveyn widget'ın içinde görünür.

Altın Kural

`new` ile nesne oluşturduğunda mutlaka bir ebeveyn ver. Bu sayede bellek yönetimini Qt'ye bırakırsın.

Bellek Yönetimi Karşılaştırması

Qt Bellek Yönetimi: Ebeveyn-Çocuk Mekanizması



Kural: Qt'da new ile oluşturulan her nesneye bir ebeveyn ata. Ebeveyn silinince çocuklar da otomatik silinir.

Bellek Yönetimi Örnekleri

```
// Parent-child ile otomatik bellek yonetimi
auto *pencere = new QWidget;            // Kok nesne
auto *buton = new QPushButton("Tikla", pencere); // cocuk
auto *etiket = new QLabel("Merhaba", pencere);   // cocuk

// pencere silindiginde buton ve etiket de silinir!
delete pencere;  // Tek delete yeterli

// ---
// destroyed() sinyali ile silme izleme
auto *sinif = new QObject;
sinif->setObjectName("BILB2000");
auto *ogr = new QObject(sinif);
ogr->setObjectName("Ali");

connect(ogr, &QObject::destroyed, [](QObject *o) {
    qDebug() << o->objectName() << "silindi";
});
delete sinif;  // "Ali silindi", "BILB2000 silindi"
```

deleteLater()

Tanım

`deleteLater()`, nesneyi hemen silmek yerine olay döngüsünün bir sonraki turunda silinmek üzere işaretler.

```
// TEHLIKELI: Slot icinde dogrudan silme
connect(buton, &QPushButton::clicked, [=]() {
    delete buton;  // TEHLIKELI! Qt silinmis nesneye erisebilir
});

// GUVENLI: deleteLater() ile ertelenmis silme
connect(buton, &QPushButton::clicked, [=]() {
    buton->deleteLater();  // Olay isleme bittikten sonra silinir
});
```

Ne Zaman Kullanılır?

Slot içinde nesne silmek gerektiğinde her zaman `deleteLater()` kullan. Sinyal zinciri tamamlandıktan sonra güvenli silme yapılır.

Smart Pointer'lar ve Qt

```
// TEHLIKELI: unique_ptr + Qt parent birlikte
auto btn = std::make_unique<QPushButton>("Tikla", pencere);
// Sorun: Hem unique_ptr hem parent silmeye calisir!
// -> Double free hatasi!

// GUVENLI: Ebeveynli nesnelerde ham isaretci
QPushButton *btn1 = new QPushButton("Tikla", pencere);
// Bellek yonetimini Qt'ye birak

// GUVENLI: Ebeveynsiz nesnelerde unique_ptr
auto btn2 = std::make_unique<QPushButton>("Bagimsiz");
// Ebeveyn yok, unique_ptr yonetir
```

Üç Kural

1. **Parent var** → ham işaretçi (`new`), Qt'ye bırak
2. **Parent yok** → `std::unique_ptr` veya stack
3. **Slot içinde silme** → `deleteLater()`

Q_PROPERTY ve Meta-Object

```
class Ogrenci : public QObject {
    Q_OBJECT
    Q_PROPERTY(QString isim READ isim WRITE setIsim NOTIFY isimDegisti)
    Q_PROPERTY(int numara READ numara WRITE setNumara NOTIFY numaraDegisti)
public:
    QString isim() const { return m_isim; }
    void setIsim(const QString &i) {
        if (m_isim != i) { m_isim = i; emit isimDegisti(i); }
    }
signals:
    void isimDegisti(const QString &yeni);
    void numaraDegisti(int yeni);
private:
    QString m_isim;
    int m_numara = 0;
};
```

Meta-Object Sorgulama

`obj->metaObject()->className()` → `"Ogrenci"` `obj->inherits("QObject")` → `true`
`obj->property("isim")` → `QVariant`

QString: Qt'nin Metin Sınıfı

Tanım

QString: Qt'nin UTF-16 tabanlı Unicode metin sınıfı. **Implicit sharing** (copy-on-write) ile verimli kopyalama.

QString vs std::string

Özellik	Fark
Kodlama	UTF-16 (her zaman)
Kopyalama	COW (ucuz)
API	arg(), split(), contains()
Null/Empty	Ayrım var

OBYS Bağlamı

Öğrenci isimleri, ders adları, hata mesajları – Qt GUI'sinde tüm metinler `QString`'dir.

Dönüşüm

`QString` → `std::string`: `toStdString()` (UTF-8'e çevirir)
`std::string` → `QString`: `QString::fromStdString()` (UTF-8 varsayar)

QString İşlemleri

```
// arg() ile formatli string
QString bilgi = QString("Ogrenci: %1, No: %2").arg(isim).arg(1001);

// split() ve join()
QStringList parcalar = "Ali,Ayse,Mehmet".split(",");
QString birlesik = parcalar.join(" | "); // "Ali | Ayse | Mehmet"

// Arama ve degistirme
bool var = metin.contains("GUI");        // true
int pos = metin.indexOf("GUI");          // 7
QString yeni = metin.replace("GUI", "Arayuz");

// Donusum
int sayi = QString("42").toInt();
double d = QString("3.14").toDouble();
QString s = QString::number(3.14, 'f', 2);

// Bosluk temizleme
QString temiz = "  Merhaba  ".trimmed();     // "Merhaba"
QString sade = "  a   b  ".simplified();     // "a b"
```

QStringList Ek Metotlar

`filter("Veri")` → "Veri" içeren elemanlar `sort()` → Alfabetik sırala `contains("X")` → Var mı?

Qt Kapsayıcıları

STL ↔ Qt Karşılıkları

STL	Qt	Açıklama
`std::vector<T>`	`QList<T>`	Dinamik dizi (Qt 6)
`std::map<K,V>`	`QMap<K,V>`	Sıralı anahtar-değer
`std::unordered_map`	`QHash<K,V>`	Hash tabanlı
`std::set<T>`	`QSet<T>`	Benzersiz küme

Implicit Sharing

```
QList<int> b = a;
```

→ Veri kopyalanmaz!

```
b.append(6);
```

→ Şimdi kopyalanır (COW)

Avantaj

Qt kapsayıcılarını fonksiyona **değer olarak** geçirmek ucuzdur – veri değişene kadar paylaşım devam eder.

Qt vs STL: Ne Zaman Hangisi?

Qt Kapsayıcısı Kullan

- Qt API'leriyle etkileşimde
- GUI veri bağlamada
- Sinyal/slot parametrelerinde
- `QList<QString>` = `QStringList`

STL Kapsayıcısı Kullan

- İç mantık ve algoritmalar
- Performans kritik kodlar
- Qt'ye bağımlı olmayan modüller
- STL algoritmaları ile doğal uyum

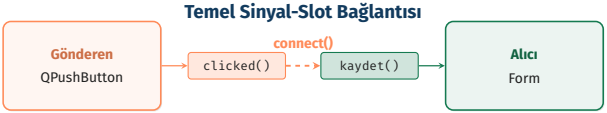
```
// QMap ile OBYS ogrenci sozlugu
QMap<int, QString> ogrenciler;
ogrenciler.insert(1001, "Ali Yilmaz");
ogrenciler[1002] = "Ayse Kaya";
QString isim = ogrenciler.value(1001);          // "Ali Yilmaz"
QString yok  = ogrenciler.value(9999, "Yok");   // "Yok"
QList<int> numaralar = ogrenciler.keys();        // [1001, 1002]
```

STL Uyumluluğu

Qt kapsayıcıları STL uyumlu iteratörlere sahiptir – `std::sort()`, `std::accumulate()` doğrudan kullanılabilir.

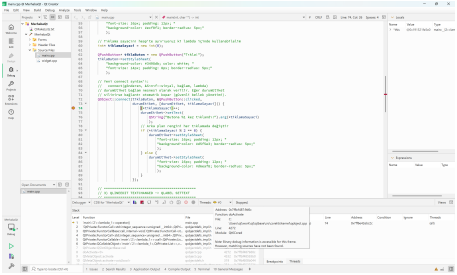
Ünite 3: Sinyaller ve Yuvalar

Sinyal/Slot Kavramı



Temel Örnek: Buton → Kapat

```
#include <QApplication>
#include <QPushButton>

int main(int argc, char *argv[]) {
    QApplication app(argc, argv);

    QPushButton buton("Tikla ve Kapat");
    buton.resize(250, 60);

    // Sinyal/slot baglantisi
    QObject::connect(
        &buton, &QPushButton::clicked,
        &app,   &QApplication::quit
    );

    buton.show();
    return app.exec();
}
```



Qt Creator'da sinyal/slot debug

Dört Parametre

`connect(gönderen, sinyal, alıcı, slot)`

Örnek: Tıklama Sayacı

```
QWidget pencere;
auto *duzen = new QVBoxLayout(&pencere);

auto *etiket = new QLabel("Tiklanma: 0");
auto *buton = new QPushButton("Tikla Beni!");
int *sayac = new int(0);  // heap'te sayac

connect(buton, &QPushButton::clicked, etiket, [=]() {
    (*sayac)++;
    etiket->setText(QString("Tiklanma: %1").arg(*sayac));
    if (*sayac % 2 == 0)
        etiket->setStyleSheet("color: teal;");
    else
        etiket->setStyleSheet("color: coral;");
});

duzen->addWidget(etiket);
duzen->addWidget(buton);
pencere.show();
```

Yaygın Yerleşik Sinyaller

Widget	Sinyal	Ne Zaman?
`QPushButton`	`clicked(bool)`	Butona tıklandığında
`QPushButton`	`toggled(bool)`	Durum değiştiğinde (checkable)
`QLineEdit`	`textChanged(QString)`	Her karakter girişinde
`QLineEdit`	`returnPressed()`	Enter tuşuna basıldığında
`QComboBox`	`currentIndexChanged(int)`	Seçim değiştiğinde
`QSpinBox`	`valueChanged(int)`	Değer değiştiğinde
`QCheckBox`	`stateChanged(int)`	İşaret durumu değiştiğinde
`QObject`	`destroyed(QObject*)`	Nesne silinirken

Parametreler Otomatik İletilir

Sinyal parametreleri bağlı slot'a otomatik olarak geçirilir. Örneğin `textChanged` sinyali yeni metin değerini slot'a iletir.

connect() Yeni Sözdizimi

Yeni (tür güvenli) – BUNU KULLAN:

```
QObject::connect(
    &buton, &QPushButton::clicked,
    &app,   &QApplication::quit
);
// Derleme zamani kontrol
// Oto. tamamlama destegi
// Refactoring guvenli
```

Eski (string tabanlı) – KULLANMA:

```
connect(buton, SIGNAL(clicked()),
        app,   SLOT(quit()));
// Calisma zamani hata
// Yazim hatasinda sessiz
//   basarisizlik
// Yuklenmis sinyalde sorun
```

Eski Syntax Neden Kötü?

SIGNAL/SLOT makroları string tabanlı. Yanlış isim yazarsan derleme geçer ama çalışma zamanında sessizce başarısız olur. Yeni syntax'ta derleyici hemen hata verir.

Lambda ile connect()

```
QPushButton *buton = new QPushButton("Kaydet");
QLabel *durum = new QLabel("Hazir");

// Lambda ile baglanti -- en esnek yontem
connect(buton, &QPushButton::clicked, [durum]() {
    durum->setText("Kaydedildi!");
    qDebug() << "Buton tiklandi, islem tamam.";
});

// Parametreli sinyal ile lambda
QLineEdit *giris = new QLineEdit;
connect(giris, &QLineEdit::textChanged,
    [durum](const QString &metin) {
        durum->setText("Yazilan: " + metin);
    });
```

Lambda Avantajları

Ayrı slot fonksiyonu yazmaya gerek yok → kısa ve okunabilir kod. Yerel değişkenleri yakalayabilir (capture).

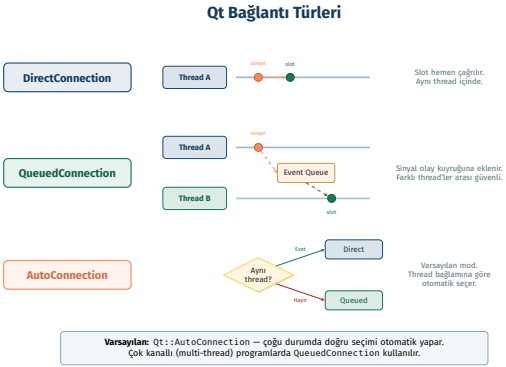
Lambda'da Context Nesnesi

```
// TEHLIKELI: Context nesnesi yok
connect(buton, &QPushButton::clicked, [etiket]() {
    etiket->setText("Tiklandi");
    // etiket silinmisse -> dangling pointer!
});

// GUVENLI: Context nesnesi ver (3. parametre)
connect(buton, &QPushButton::clicked, etiket, [etiket]() {
    etiket->setText("Tiklandi");
    // etiket silinirse baglanti otomatik kopar
});
```

Önemli

Lambda ile `connect` kullanırken **3. parametre olarak context nesnesi** ver. Bu nesne silindiğinde bağlantı otomatik kopar ve dangling pointer sorunu önlenir.

Bağlantı Türleri



Tür	Davranış	Kullanım
`AutoConnection`	Aynı thread → Direct, farklı → Queued	Varsayılan
`DirectConnection`	Slot hemen çağrılır (senkron)	Performans
`QueuedConnection`	Slot olay döngüsünde çağrılır (asenkron)	Thread-safe
`UniqueConnection`	Aynı bağlantı tekrarlanmaz	Güvenlik

Bağlantı Türü Belirtme

```
// Varsayilan: AutoConnection (cogu durumda yeterli)
QObject::connect(btn, &QPushButton::clicked,
                 this, &Form::kaydet);

// Acik tur belirtme (5. parametre)
QObject::connect(btn, &QPushButton::clicked,
                 this, &Form::kaydet,
                 Qt::DirectConnection);

// Unique + Queued birlikte (OR ile birlestir)
QObject::connect(timer, &QTimer::timeout,
                 this, &Form::guncelle,
                 Qt::QueuedConnection | Qt::UniqueConnection);
```

Ne Zaman Belirtmeli?

Çoğu durumda `AutoConnection` yeterlidir. Farklı thread'ler arası iletişimde `QueuedConnection` açıkça belirtmek güvenlidir. (Bölüm 9–11'de detaylı göreceğiz.)

disconnect()

```
// Yontem 1: Tam parametreli kesme
QObject::disconnect(&buton, &QPushButton::clicked,
                    &app, &QApplication::quit);

// Yontem 2: Bir nesnenin TUM baglantilarini kes
buton.disconnect();

// Yontem 3: Baglanti nesnesi ile kontrol
QMetaObject::Connection bag = connect(
    &buton, &QPushButton::clicked, []() { /*...*/ });
// Daha sonra kes:
QObject::disconnect(bag);
```

Otomatik Temizleme

Gönderen veya alıcı nesne silinirse bağlantı **otomatik kopar**. Çoğu durumda elle `disconnect()` gerekmez.

Özel Sinyal Tanımlama

```
class OgrenciFormu : public QObject {
    Q_OBJECT
public:
    explicit OgrenciFormu(QObject *parent = nullptr);

signals:   // Sinyaller -- sadece BILDIRIM, govde YAZILMAZ
    void kaydetTiklandi(const QString &isim, int numara);
    void formTemizlendi();
    void hataOlustu(const QString &mesaj);
};
```

Sinyal Kuralları:

1. Gövdesi yazılmaz – moc otomatik üretir
2. Her zaman `void` döndürür
3. Her zaman `public` erişime sahiptir
4. Parametre alabilir

Özel Slot Tanımlama

```
class OgrenciFormu : public QObject {
    Q_OBJECT
    // ... signals: (yukardaki gibi) ...

public slots:   // Herkes baglayabilir
    void kaydet() {
        if (m_isim.isEmpty()) {
            emit hataOlustu("Isim bos olamaz!");
            return;
        }
        emit kaydetTiklandi(m_isim, m_numara);
    }

protected slots:  // Alt siniflar baglayabilir
    void dogrula();

private slots:    // Sadece bu sinif baglayabilir
    void dahiliGuncelle();

private:
    QString m_isim;
    int m_numara = 0;
};
```

emit Anahtar Kelimesi

```
void OgrenciFormu::kaydet() {
    // Veri dogrulamasi yap
    if (m_isim.isEmpty()) {
        emit hataOlustu("Isim bos olamaz!");
        return;
    }
    // Basarili kayit sinyali
    emit kaydetTiklandi(m_isim, m_numara);
}

// Baglanti ornegi
OgrenciFormu *form = new OgrenciFormu;
OgrenciListesi *liste = new OgrenciListesi;
connect(form, &OgrenciFormu::kaydetTiklandi,
        liste, &OgrenciListesi::ogrenciEkle);
```

emit ne yapar?

`emit` boş bir makrodur (`#define emit`). Kodun okunabilirliğini artırır: "burada sinyal yayınlıyorum" mesajı verir. Teknik olarak `emit f()` ile `f()` arasında fark yoktur.

Parametreli Sinyal/Slot

```
// Sinyal: 2 parametre
signals: void veriGeldi(int id, QString mesaj);

// GECERLI: Slot 2 parametre alir
void isle(int id, QString mesaj);

// GECERLI: Slot 1 parametre alir (mesaj goz ardi edilir)
void isle(int id);

// GECERLI: Slot parametre almaz (ikisi de goz ardi edilir)
void isle();

// GECERSIZ: Slot sinyalden FAZLA parametre ALAMAZ
// void isle(int id, QString mesaj, double ekstra);
```

Parametre Uyum Kuralı

Sinyalin parametre sayısı $\geq$ slot'un parametre sayısı olmalı. Slot, sinyalden daha az parametre alabilir – fazla parametreler göz ardı edilir. Ancak slot, sinyalden **fazla** parametre **alamaz**!

OBYS'de Sinyal Kullanımı

Fan-out: Bir sinyal, birden fazla hedefe bağlanabilir:

```
// Tek sinyal -> uc hedef
connect(form, &OgrenciFormu::kaydetTiklandi,
        veritabani, &VeritabaniYoneticisi::kaydet);

connect(form, &OgrenciFormu::kaydetTiklandi,
        this, [](const QString &isim, int no) {
            qDebug() << "Kaydedildi:" << isim << no;
        });

connect(form, &OgrenciFormu::kaydetTiklandi,
        istatistik, &IstatistikModulu::kayitSayisiniArtir);
```

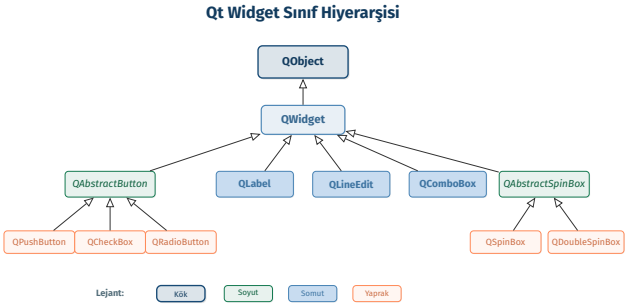
Sinyal Zincirleme

Bir sinyal başka bir sinyale de bağlanabilir: `connect(form, &Form::kaydet, this, &This::islemTamam)`

Sinyal → sinyal → slot zinciri oluşturabilirsin.

Ünite 4: Temel Widget'lar

Widget Hiyerarşisi



Widget Nedir?

Widget = pencere bileşeni. Her buton, etiket, metin kutusu bir widget'tır. Tüm widget'lar `QWidget` sınıfından türer, o da `QObject`'ten türer.

`QObject` → `QWidget` → `QPushButton`, `QLabel`, `QLineEdit`, `QComboBox`, `QSpinBox`, ...

QPushButton

```
#include <QPushButton>

// Metin ile buton
QPushButton *kaydet = new QPushButton("Kaydet", parent);

// Ikon ve metin ile buton
QPushButton *sil = new QPushButton(QIcon("sil.png"), "Sil", parent);

// Buton ozellikleri
kaydet->setEnabled(false);      // Devre disi birak
kaydet->setToolTip("Formu kaydet");    // Ipucu metni
kaydet->setFixedSize(120, 40);         // Sabit boyut
```

Temel Sinyaller:

- `clicked(bool checked)` – tıklandığında
- `pressed()` / `released()` – basılı/bırakıldığında
- `toggled(bool)` – durum değiştiğinde (checkable butonlarda)

QLabel

```
#include <QLabel>

// Duz metin
QLabel *baslik = new QLabel("OBYS Giris Ekrani", parent);

// HTML formatli metin
QLabel *zengin = new QLabel(
    "<h2 style='color:teal'>Hosgeldiniz</h2>"
    "<p>Ogrenci Bilgi Yonetim Sistemi</p>", parent);
zengin->setTextFormat(Qt::RichText);

// Hizalama
baslik->setAlignment(Qt::AlignCenter);

// Kelime sarma
baslik->setWordWrap(true);
```

Önemli: `setText(str)`, `setPixmap(px)`, `setAlignment(flags)`, `setWordWrap(bool)`

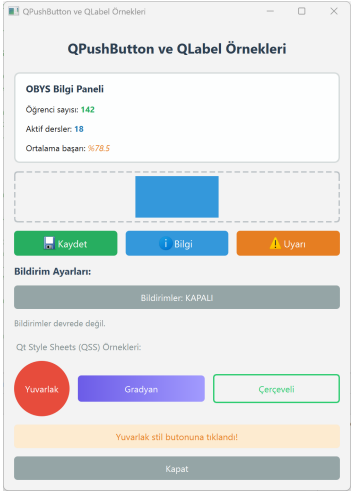
Qt Style Sheets

```
// CSS benzeri stil uygulama
buton->setStyleSheet(
    "QPushButton {"
    "    background-color: #008080;"
    "    color: white;"
    "    border-radius: 6px;"
    "    padding: 8px 16px;"
    "    font-weight: bold;"
    "}"
    "QPushButton:hover {"
    "    background-color: #009999;"
    "}"
    "QPushButton:pressed {"
    "    background-color: #006666;"
    "}"
);
```

İpucu: `setStyleSheet()` tüm alt widget'ları da etkiler. Widget'a özel stil için `objectName` ile seçici kullan: `"#kaydetBtn { ... }"`

Buton ve Etiket Birlikte

```
#include <QApplication>  // + QPushButton, QLabel
#include <QVBoxLayout>   // + QWidget

int main(int argc, char *argv[]) {
    QApplication app(argc, argv);
    QWidget pencere;
    auto *duzen = new QVBoxLayout(&pencere);
    auto *etiket = new QLabel("Hosgeldiniz!");
    auto *buton = new QPushButton("Tikla Beni");

    etiket->setStyleSheet(
        "font-size: 18px; color: teal;");
    buton->setStyleSheet(
        "padding: 8px; background: #008080;"
        "color: white; border-radius: 4px;");

    duzen->addWidget(etiket);
    duzen->addWidget(buton);
    connect(buton, &QPushButton::clicked,
        [etiket]() {
        etiket->setText("Butona basildi!");
    });
    pencere.show(); return app.exec();
}
```



QLineEdit

```
#include <QLineEdit>

auto *isim = new QLineEdit(parent);
isim->setPlaceholderText("Adinizi girin...");
isim->setMaxLength(50);

// Sifre modu
auto *sifre = new QLineEdit(parent);
sifre->setEchoMode(QLineEdit::Password);

// Anlik metin degisikligini yakala
connect(isim, &QLineEdit::textChanged,
    [=](const QString &metin) {
        etiket->setText("Merhaba, " + metin + "!");
    });

// Enter tusu ile kaydet
connect(isim, &QLineEdit::returnPressed,
    [=]() { qDebug() << "Kaydedildi:" << isim->text(); });
```

Sinyaller: `textChanged` `textEdited` `returnPressed` `editingFinished`

QComboBox

```
#include <QComboBox>

auto *bolum = new QComboBox(parent);
bolum->addItem("Bilgisayar Bilimleri");
bolum->addItem("Matematik");
bolum->addItem("Fizik");

// Toplu ekleme
bolum->addItems({"Kimya", "Biyoloji", "Istatistik"});

// Mevcut secimi al
int index = bolum->currentIndex();        // 0, 1, 2, ...
QString metin = bolum->currentText();     // "Bilgisayar Bilimleri"

// Secim degisince
connect(bolum, &QComboBox::currentTextChanged,
    [=](const QString &s) {
        qDebug() << "Secilen bolum:" << s;
    });
```

Sinyaller: `currentIndexChanged(int)` `currentTextChanged(QString)`

QSpinBox ve QDoubleSpinBox

QSpinBox (Tam sayı):

```
auto *yas = new QSpinBox;
yas->setRange(17, 65);
yas->setValue(20);
yas->setSuffix(" yas");
yas->setPrefix("Yas: ");
yas->setSingleStep(1);

connect(yas,
    &QSpinBox::valueChanged,
    [](int v) {
    qDebug() << "Yas:" << v;
});
```

QDoubleSpinBox (Ondalıklı):

```
auto *ort = new QDoubleSpinBox;
ort->setRange(0.0, 4.0);
ort->setValue(2.5);
ort->setSuffix(" GNO");
ort->setSingleStep(0.01);
ort->setDecimals(2);

connect(ort,
    &QDoubleSpinBox::valueChanged,
    [](double v) {
    qDebug() << "GNO:" << v;
});
```

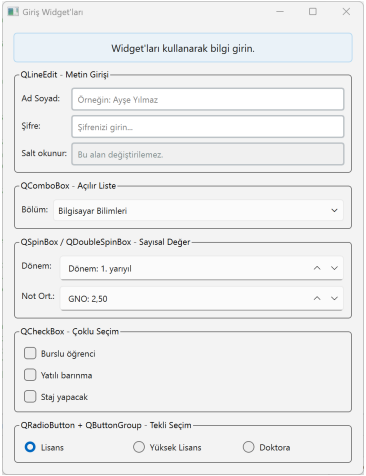
QCheckBox ve QRadioButton

QCheckBox: (çoklu seçim)

```
auto *burs = new QCheckBox(
    "Burslu mu?");
auto *yurt = new QCheckBox(
    "Yurtta mi?");
// Her ikisi de secilebilir
```

QRadioButton: (tekli seçim)

```
auto *lisans = new QRadioButton(
    "Lisans");
auto *yuksek = new QRadioButton(
    "Y. Lisans");
lisans->setChecked(true);
// Sadece biri secili
```



Layout Yöneticileri

Tanım

Layout (düzen yöneticisi), widget'ları pencere içinde otomatik olarak konumlandıran ve boyutlandıran mekanizmadır. Pencere boyutu değiştiğinde widget'lar otomatik yeniden düzenlenir.

Dört Temel Layout Türü:

1. `QVBoxLayout` – Widget'ları **dikey** (alt alta) dizer
2. `QHBoxLayout` – Widget'ları **yatay** (yan yana) dizer
3. `QGridLayout` – Widget'ları **ızgara** (satır/sütun) düzeninde yerleştirir
4. `QFormLayout` – **Etiket-widget** çiftlerini hizalar (form düzeni)

Neden Layout?

Sabit piksel koordinatı kullanma! Pencere boyutu değiştiğinde widget'lar bozulur. Layout ile otomatik yeniden boyutlanma sağlanır.

QVBoxLayout ve QHBoxLayout

Dikey düzen:

```
QWidget pencere;
auto *vbox = new QVBoxLayout(
    &pencere);

vbox->addWidget(
    new QLabel("Baslik"));
vbox->addWidget(
    new QLineEdit);
vbox->addWidget(
    new QPushButton("Tamam"));
// Ust -> alt sirayla
```

Yatay düzen:

```
auto *hbox = new QHBoxLayout;

hbox->addWidget(
    new QPushButton("Kaydet"));
hbox->addWidget(
    new QPushButton("Iptal"));
hbox->addWidget(
    new QPushButton("Sil"));
// Sol -> sag sirayla

// ic ice layout
vbox->addLayout(hbox);
```

İç İçe Layout

Layout'lar iç içe yerleştirilebilir. `addLayout()` ile bir layout başka bir layout'a eklenir. Bu sayede karmaşık düzenler oluşturulur.

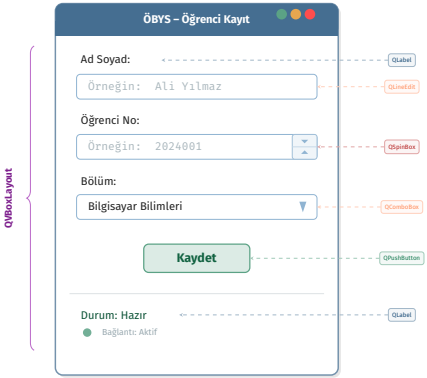
QGridLayout ve QFormLayout

```
// QGridLayout: Izgara duzeni (satir, sutun)
auto *grid = new QGridLayout;
grid->addWidget(new QLabel("Isim:"),     0, 0);  // satir 0, sutun 0
grid->addWidget(new QLineEdit,           0, 1);  // satir 0, sutun 1
grid->addWidget(new QLabel("Numara:"),   1, 0);
grid->addWidget(new QSpinBox,            1, 1);
grid->addWidget(new QPushButton("Kaydet"), 2, 0, 1, 2); // 2 sutun kapla

// QFormLayout: Etiket-widget cifti (form duzeni)
auto *form = new QFormLayout;
form->addRow("Ad Soyad:",  new QLineEdit);
form->addRow("Numara:",    new QSpinBox);
form->addRow("Bolum:",     new QComboBox);
form->addRow("",           new QCheckBox("Burslu"));
```

Hangisini Seçmeli?

QFormLayout: Basit formlar için (etiket + widget) **QGridLayout**: Karmaşık ızgara düzenleri için

OBYS Kayıt Formu: Tasarım



Form Alanları:

- **Ad Soyad**: `QLineEdit` – metin girişi
- **Öğrenci No**: `QSpinBox` – 1000–9999
- **Bölüm**: `QComboBox` – açılır liste
- **Burslu**: `QCheckBox` – evet/hayır
- **Kaydet**: `QPushButton` – form gönderme
- **Temizle**: `QPushButton` – sıfırlama
- **Durum**: `QLabel` – geri bildirim

Düzen

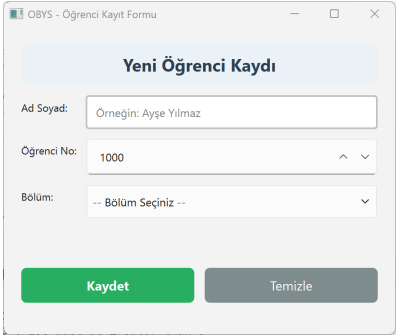
`QFormLayout` ile etiket-widget çiftleri
`QHBoxLayout` ile butonlar yan yana

OBYS Formu: Widget Oluşturma

```
QWidget pencere;
pencere.setWindowTitle("OBYS Ogrenci Kayit");
auto *anaDuzen = new QVBoxLayout(&pencere);
auto *form = new QFormLayout;

auto *isim = new QLineEdit;
isim->setPlaceholderText("Ad Soyad girin");
isim->setMaxLength(50);

auto *numara = new QSpinBox;
numara->setRange(1000, 9999);

auto *bolum = new QComboBox;
bolum->addItems({"Bilgisayar Bil.", "Matematik", "Fizik"});

auto *burs = new QCheckBox("Burslu");

form->addRow("Isim:", isim);
form->addRow("Numara:", numara);
form->addRow("Bolum:", bolum);
form->addRow("", burs);

auto *kaydet = new QPushButton("Kaydet");
auto *temizle = new QPushButton("Temizle");
anaDuzen->addLayout(form);
```

OBYS Formu: Doğrulama

```
QLabel *durum = new QLabel;
anaDuzen->addWidget(durum);
connect(kaydet, &QPushButton::clicked, [=]() {
    if (isim->text().trimmed().isEmpty()) {
        durum->setText("Hata: Isim bos!");
        durum->setStyleSheet("color: red;");
        isim->setFocus();  return;
    }
    QString mesaj = QString("Kaydedildi: %1 (%2) - %3")
        .arg(isim->text()).arg(numara->value())
        .arg(bolum->currentText());
    durum->setText(mesaj);
    durum->setStyleSheet("color: green;");
    QMessageBox::information(&pencere, "Basarili", mesaj);
});
connect(temizle, &QPushButton::clicked, [=]() {
    isim->clear();  numara->setValue(1000);
    bolum->setCurrentIndex(0);
    burs->setChecked(false);  durum->clear();
});
```



Bölüm 4: Büyük Resim

Ünite 1: Qt Ekosistemi

Qt 6 + CMake + C++20 ortamı kurulumu, modüler yapı, ilk uygulama

Ünite 2: QObject

Meta-object sistemi, parent-child sahiplik, bellek yönetimi, Q_PROPERTY

Ünite 3: Sinyal/Slot

Tür güvenli connect, lambda, bağlantı türleri, özel sinyal/slot

Ünite 4: Widget'lar

Buton, etiket, giriş widget'ları, layout yöneticileri, OBYS formu

OBYS İlerleme

Bölüm 1–3: STL ile veri katmanı ✓ **Bölüm 4**: İlk GUI formu ✓ **Sonraki**: QMainWindow, menü, tablo

Bu Hafta Ne Öğrendik?

Qt Temelleri:

- Qt 6, CMake, modüler yapı
- QApplication ve olay döngüsü
- Qt Creator ve `find_package`
- Sınıf bazlı vs modül bazlı include

QObject ve Bellek:

- Q_OBJECT makrosu, moc
- Parent-child sahiplik modeli
- deleteLater(), smart pointer kuralları
- Q_PROPERTY, meta-object sorgulama

Sinyal/Slot:

- Yeni connect() sözdizimi
- Lambda ile bağlantı, context nesnesi
- Bağlantı türleri (Auto/Direct/Queued)
- Özel sinyal/slot, emit, disconnect
- Parametre uyumu, fan-out, zincirleme

Widget'lar:

- QPushButton, QLabel, QLineEdit
- QComboBox, QSpinBox, QCheckBox
- Layout yöneticileri (VBox/HBox/Grid/Form)
- Qt Style Sheets

OBYS'de Her Şey Bir Arada

```
#include <QApplication>   // + QWidget, QFormLayout, QLineEdit
#include <QSpinBox>        // + QComboBox, QPushButton, QLabel

int main(int argc, char *argv[]) {
    QApplication app(argc, argv);
    QWidget pencere;  pencere.setWindowTitle("OBYS Kayit Formu");
    auto *form = new QFormLayout(&pencere);
    auto *isim = new QLineEdit;   isim->setPlaceholderText("Ad Soyad");
    auto *numara = new QSpinBox;  numara->setRange(1000, 9999);
    auto *bolum = new QComboBox;
    bolum->addItems({"Bilg. Bil.", "Matematik", "Fizik"});
    auto *durum = new QLabel;
    form->addRow("Isim:", isim);  form->addRow("No:", numara);
    form->addRow("Bolum:", bolum);
    auto *btn = new QPushButton("Kaydet");
    form->addRow(btn);  form->addRow(durum);
    QObject::connect(btn, &QPushButton::clicked, [&]() {
        if (isim->text().isEmpty()) {
            durum->setText("Isim bos!"); durum->setStyleSheet("color:red;");
        } else {
            durum->setText("Kaydedildi: " + isim->text());
            durum->setStyleSheet("color:green;");
        }
    });
    pencere.show();  return app.exec();
}
```

Gelecek Hafta: Qt Pencere ve Düzen Yönetimi

Bölüm 5'te neler göreceğiz:

- `QMainWindow`: menü çubuğu, araç çubuğu, durum çubuğu
- `QDialog`: modal ve modeless diyaloglar
- `QTabWidget` ile sekmeli arayüz
- `QStackedWidget` ile sayfa geçişleri
- `QAction` ve kısayol tuşları
- OBYS: Tam işlevli ana pencere tasarımı

Hazırlık

Kitabın Bölüm 5'ini okuyun. Qt Creator ile basit bir form uygulaması yapın. Bu haftaki OBYS kayıt formunu derleyip çalıştırmayı deneyin.